Despliegue de un proyecto Django con Apache2 detrás de nginx (Docker) en EC2

Guía paso a paso para desplegar un proyecto Django **sin base de datos** usando **Apache2 + mod_wsgi** en una instancia EC2 que ya tiene **otro stack en Docker (nginx + backend + frontend)**, reutilizando el nginx de Docker como reverse proxy y SSL central.

Arquitectura final

```
Internet → :80/:443 nginx (Docker)
              ├── pidelibre.com → contenedores Docker (Django/React)
              └── sruedadev.com → host.docker.internal:8080 → Apache2 → Django
(venv)
```

- nginx Docker maneja **todo el tráfico público** (puertos 80 y 443) y los certificados SSL.
- Apache2 corre **en el host**, escuchando en `*:8080`, sirviendo Django vía `mod_wsgi`.
- nginx hace `proxy_pass` a `host.docker.internal:8080` para llegar al Apache del host.

Paso 1 — Route 53: apuntar el dominio

En la hosted zone del dominio en Route 53, crear:

- Registro **A**: `dominio.com` → IP pública de la EC2
- Registro **A**: `www.dominio.com` → IP pública de la EC2

Verificar propagación:

```
dig dominio.com +short
```

Paso 2 — Security Group de la EC2

Asegurar que estén abiertos al mundo:

- **22** (SSH)
- **80** (HTTP)
- **443** (HTTPS)

No abrir el 8080 — será solo interno.

Paso 3 — Instalar Apache2, mod_wsgi y Python

```
sudo apt update
sudo apt install -y apache2 libapache2-mod-wsgi-py3 python3-pip python3-venv git
```

Paso 4 — Clonar el repositorio

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/usuario/proyecto.git
sudo chown -R ubuntu:ubuntu /var/www/proyecto
cd /var/www/proyecto
```

Paso 5 — Crear virtualenv e instalar dependencias

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt   # o `pip install django` si no existe
deactivate
```

Paso 6 — Ajustar `settings.py`

Editar el `settings.py` del proyecto:

```
DEBUG = False

ALLOWED_HOSTS = [
    'dominio.com',
    'www.dominio.com',
    '127.0.0.1',
    'localhost',
]

# CSRF para formularios detrás de proxy HTTPS
CSRF_TRUSTED_ORIGINS = [
    'https://dominio.com',
    'https://www.dominio.com',
]

# Django respeta el esquema HTTPS pasado por el proxy
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
USE_X_FORWARDED_HOST = True

import os
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
STATIC_URL = '/static/'
```

Paso 7 — Recolectar estáticos

```
cd /var/www/proyecto
source venv/bin/activate
python manage.py collectstatic --noinput
deactivate
```

Paso 8 — Permisos para Apache

```
sudo chown -R www-data:www-data /var/www/proyecto
sudo chmod -R 755 /var/www/proyecto
```

Paso 9 — Apache escucha en el puerto 8080

Editar `/etc/apache2/ports.conf`:

```
sudo nano /etc/apache2/ports.conf
```

Cambiar `Listen 80` por:

```
Listen 8080
```

(Mantener el bloque `Listen 443` dentro de `<IfModule ssl_module>` como esté.)

Paso 10 — Crear el VirtualHost de Django

Identificar primero la carpeta real del proyecto Django (la que contiene `wsgi.py`):

```
find /var/www/proyecto -name "wsgi.py"
```

Suponiendo que sale `/var/www/proyecto/miapp/wsgi.py`, crear:

```
sudo nano /etc/apache2/sites-available/miproyecto.conf
```

Contenido:

```
<VirtualHost *:8080>
    ServerName dominio.com
    ServerAlias www.dominio.com

    Alias /static/ /var/www/proyecto/staticfiles/
```

```
    <Directory /var/www/proyecto/staticfiles>
        Require all granted
    </Directory>

    <Directory /var/www/proyecto/miapp>
        <Files wsgi.py>
            Require all granted
        </Files>
    </Directory>

    WSGIDaemonProcess miapp python-home=/var/www/proyecto/venv python-
path=/var/www/proyecto
    WSGIProcessGroup miapp
    WSGIScriptAlias / /var/www/proyecto/miapp/wsgi.py

    ErrorLog ${APACHE_LOG_DIR}/miproyecto_error.log
    CustomLog ${APACHE_LOG_DIR}/miproyecto_access.log combined
</VirtualHost>
```

> ⚠ Cuidado: la carpeta del proyecto (la que contiene `wsgi.py`) NO siempre se llama igual que el repo. Verificar siempre con `find`.

Habilitar el sitio, deshabilitar el default y reiniciar:

```
sudo a2ensite miproyecto.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl restart apache2
```

Verificar:

```
sudo apache2ctl -S
sudo ss -tlnp | grep apache
curl -v -H "Host: dominio.com" http://127.0.0.1:8080/
```

Debería responder con HTML de Django. Si da 404, revisar `/var/log/apache2/miproyecto_error.log`.

Paso 11 — nginx Docker como reverse proxy

11.1 Asegurar `extra_hosts` en `docker-compose.prod.yaml`

En el servicio nginx, agregar (si no está):

```
nginx:
  # ...
  extra_hosts:
    - "host.docker.internal:host-gateway"
```

Esto hace que `host.docker.internal` resuelva a la IP del host desde dentro del contenedor en Linux.

11.2 Agregar el server block en `nginx/prod.conf`

Inicialmente solo HTTP (luego se cambiará a HTTPS):

```
server {
    listen 80;
    server_name dominio.com www.dominio.com;

    location / {
        proxy_pass http://host.docker.internal:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

11.3 Rebuild del contenedor nginx

Como `prod.conf` se copia dentro de la imagen (`COPY prod.conf /etc/nginx/conf.d/`), hay que hacer **rebuild**, no solo restart:

```
cd ~/pidelibre
docker compose -f docker-compose.prod.yaml up -d --build --force-recreate nginx
```

11.4 Verificar el puente

```
# host.docker.internal debe resolver
docker exec pidelibre-nginx getent hosts host.docker.internal

# Debe devolver el HTML de Django
docker exec pidelibre-nginx curl -s -H "Host: dominio.com"
http://host.docker.internal:8080/ | head -20
```

Probar desde el navegador: `http://dominio.com` → debería cargar el sitio.

Paso 12 — Emitir certificado SSL con Let's Encrypt

Como nginx ocupa el puerto 80, usar `certbot --standalone` deteniendo nginx temporalmente:

```
# 1. Instalar certbot en el host
sudo apt install -y certbot
```

```
# 2. Detener nginx
cd ~/pidelibre
docker compose -f docker-compose.prod.yaml stop nginx

# 3. Emitir certificado
sudo certbot certonly --standalone \
  -d dominio.com -d www.dominio.com \
  --email tu-email@ejemplo.com --agree-tos --no-eff-email

# 4. Levantar nginx
docker compose -f docker-compose.prod.yaml start nginx
```

Verificar archivos creados:

```
sudo ls /etc/letsencrypt/live/dominio.com/
# README  cert.pem  chain.pem  fullchain.pem  privkey.pem
```

Como `/etc/letsencrypt` ya está montado en el contenedor nginx por volumen, los certificados quedan disponibles automáticamente dentro del contenedor.

Paso 13 — Activar HTTPS en nginx

Editar `nginx/prod.conf` y **reemplazar** el server block HTTP simple del paso 11.2 por estos dos:

```
# Redirige HTTP → HTTPS
server {
    listen 80;
    server_name dominio.com www.dominio.com;
    return 301 https://$host$request_uri;
}

# HTTPS → proxy a Apache
server {
    listen 443 ssl;
    server_name dominio.com www.dominio.com;

    ssl_certificate     /etc/letsencrypt/live/dominio.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/dominio.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://host.docker.internal:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

Rebuild nginx:

```
cd ~/pidelibre
docker compose -f docker-compose.prod.yaml up -d --build --force-recreate nginx
```

Probar:

```
curl -vI https://dominio.com 2>&1 | grep -E "HTTP|subject|issuer|expire"
```

Y abrir `https://dominio.com` en el navegador. Candado verde 🔒 = listo.

Troubleshooting frecuente

Apache devuelve 404 default

- `apache2ctl -S` no muestra el VirtualHost esperado → verificar `a2ensite` y `ports.conf`.
- `Target WSGI script not found` en el log → la ruta de `WSGIScriptAlias` no coincide con la carpeta real del `wsgi.py`. Correr `find /var/www/proyecto -name "wsgi.py"` y corregir.
- Mismatch `127.0.0.1:8080` en VirtualHost vs `Listen 8080` global → usar `*:8080` en el VirtualHost.

nginx no llega a Apache

- `docker exec nginx getent hosts host.docker.internal` no resuelve → falta `extra_hosts` o no se hizo recreate del contenedor.
- `connection refused` → Apache no escucha en todas las interfaces. Verificar con `ss -tlnp | grep apache` que sea `*:8080` y no `127.0.0.1:8080`.
- Cambios en `prod.conf` no se aplican → la imagen lo copia con `COPY`, hay que `--build`, no solo `restart`.

Sitio carga pero "no es seguro"

- Verificar que la URL en el navegador sea `https://` y no `http://`.
- F12 → Security → ver causa exacta.
- Errores `ERR_BLOCKED_BY_CLIENT` en consola = ad blocker bloqueando pixels de Google Ads, **no afectan la seguridad**.
- `xmlns="http://www.w3.org/2000/svg"` en grep = namespace XML, no es mixed content real.

Formularios devuelven 403 CSRF

- Falta `CSRF_TRUSTED_ORIGINS` y `SECURE_PROXY_SSL_HEADER` en `settings.py` (paso 6).

Comandos útiles del día a día

```
# Reiniciar Apache (después de cambiar código Django o settings)
sudo systemctl restart apache2

# Ver logs de Apache
sudo tail -f /var/log/apache2/miproyecto_error.log

# Recolectar estáticos después de cambios
cd /var/www/proyecto && source venv/bin/activate && python manage.py collectstatic
--noinput && deactivate

# Pull de cambios desde git
cd /var/www/proyecto && git pull && sudo systemctl restart apache2

# Rebuild nginx tras cambios en prod.conf
cd ~/pidelibre && docker compose -f docker-compose.prod.yaml up -d --build --
force-recreate nginx

# Renovar certificados (auto cada 60 días con cron, o manual)
sudo certbot renew
```